

Ultrasonic to LoRa Bench Test

Two ESP32 boards on a desk. One measures distance and transmits it; the other receives and flashes an LED. No rain gauge, no solar, no server.

This is the test to build first. It proves the radio link, the SPI wiring and the ultrasonic sensor in one go. Everything transfers to the field build: the pin map is identical and the packet is the same 13-byte format, so the real gateway firmware will decode these packets unchanged. The sketches are tuned for a **433 MHz RA-02 carrying an SX1278**.

What you need

PART	QTY	NOTE
ESP32-WROOM-32 DevKit V1	1	The sender
ESP32-C3 board (SuperMini or DevKitM-1)	1	The receiver — different pin numbers, see page 3
RA-02 LoRa module (SX1278)	2	433 MHz. Fit 433 MHz antennas before powering up
JSN-SR04T v2.0 ultrasonic	1	v2.0 runs on 3.3 V; a v1.0 needs 5 V and a divider on ECHO
LED + 330 Ω resistor	1	Optional — the DevKit's onboard LED does the same job
Breadboard and jumper wires	–	Male-to-female for the module headers
18650 Li-ion cell + holder	2	Or any single lithium cell, 3.0–4.2 V
HT7333-A regulator	2	TO-92, 3.3 V, 100 mV dropout. Not an AMS1117
100 μ F + 470 μ F capacitors	2 each	Regulator output, and at the LoRa module's pads
Slide switch	2	So the battery can be cut while USB is connected
USB cables	2	For flashing and the serial monitor only

What is different from the field build

FIELD VERSION	BENCH VERSION	WHY
Tipping bucket, DS18B20, battery divider	all removed	No rain gauge yet; sound speed fixed at 25 °C; USB or cell, unmonitored
Deep sleep, 5-minute cycle	awake, 2-second cycle	You want to see it working, not wait
Solar panel and CN3065 charger	removed	The cell and HT7333 stay — see page 5
Gateway: classic ESP32, WiFi, MQTT, SD	ESP32-C3, LED and serial	Nothing to configure, nothing to go wrong

Which module you have decides the frequency

Two modules look almost identical and are both sold as "RA-02". The chip on the board decides which band it can reach, and no amount of code will move it outside that band.

CHIP	TUNING RANGE	USE	SKETCH SETTING
SX1278	137–525 MHz	433 MHz — the module you have	433000000L
SX1276	137–1020 MHz	865–867 MHz India LPWAN band	865100000L

Both sketches here are set to 433000000L for the SX1278. Only this one constant changes between the two chips.

USE THE ANTENNA THAT MATCHES THE BAND

A 433 MHz whip is noticeably longer than an 868 MHz one. Fitting the wrong one leaves the radio badly mismatched: it will transmit, you may even receive at a metre, and the range collapses to almost nothing. Check that the antennas that came with these modules are the 433 MHz ones.

433 MHZ VERSUS 865 MHZ FOR THE REAL DEPLOYMENT

433 MHz is fine for this bench test. For the field system check the current WPC and ETA rules first: 865–867 MHz is the band delicensed for LPWAN at 1 W ERP, while 433 MHz is more restricted and shares space with car keys and telemetry. Deployed nodes at 865 MHz need SX1276 modules — but keep these boards, the rig works at either frequency.

THE ONE RULE THAT HAS NOT CHANGED

Never power a LoRa module with the antenna unscrewed. The reflected power destroys the output stage, usually on the first transmission. Fit both antennas before either board sees power.

KEEP THE 470 MF CAPACITOR

Put 470 μ F across the LoRa module's own VCC and GND pads on both boards. Breadboard contacts have more resistance than solder, so the 120 mA transmit burst browns out the ESP32 more easily here than on a soldered board.

1 • Sender wiring

ESP32, LoRa module, ultrasonic sensor. Ten wires.

Bench sender – ESP32 + ultrasonic + LoRa

Battery powered - no USB needed to run.
Switch the battery OFF before plugging USB in.
Blue LED on GPIO2 blinks once per transmission.

18650 Li-ion cell
3.7 V nominal, 3.0-4.2 V

ON / OFF switch

HT7333-A
3.3 V low-dropout

C2 100 uF

Battery negative and the regulator ground are one net.

ESP32

DEVKIT V1

RA-02
SX1278 433 MHz SF9
BW125 sync 8x34

JSN-SR04T
v2.0 - runs on 3.3 V

antenna - fit before power

point it at the floor

USB - power and serial monitor

3.3 V GND NSS / LED SCK MISO MOSI RESET DIO0 TRIG ECHO

ESP32 PIN	GOES TO	WIRE	NOTE
3V3	HT7333 OUT, LoRa VCC, JSN VCC	red	Power comes IN here — see page 5
GND	Battery —, HT7333 GND, LoRa GND, JSN GND	black	One common ground rail
VIN	nothing	—	A 3.7 V cell cannot drive the onboard regulator
D5	LoRa NSS	orange	Chip select
D18	LoRa SCK	yellow	SPI clock
D19	LoRa MISO	green	—
D23	LoRa MOSI	blue	Bottom of the right header
D14	LoRa RST	purple	On the left header
D26	LoRa DIO0	pink	On the left header
TX2 • GPIO17	JSN TRIG	white	—
RX2 • GPIO16	JSN ECHO	brown	Direct on v2.0 only
GPIO2	onboard LED	—	Already wired on the DevKit

IF YOUR JSN-SR04T IS A V1.0 BOARD

It wants 5 V, and its ECHO pin then swings to 5 V, which will damage the ESP32 input. Power it from the DevKit's 5 V (VIN) pin instead, and put a 10 kΩ / 15 kΩ divider between ECHO and GPIO16 — 10 kΩ from ECHO to the junction, 15 kΩ from the junction to ground, GPIO16 on the junction. TRIG is fine at 3.3 V.

2 • Receiver wiring

An ESP32-C3 this time — different pin numbers, same radio, same packet. The sender stays a classic ESP32.

Bench receiver — ESP32-C3 + LoRa + LED

Battery powered - no USB needed to run.
Switch the battery OFF before USB in.

Pin names are the SuperMini's. Other C3 boards use the same GPIO numbers in a different physical order - go by the number, not the position.

Onboard LED on GP8 is ACTIVE LOW and flashes in step with this one.

No ultrasonic, no SD card, no WiFi.

Battery negative and the regulator ground are one net.

C3 PIN	GOES TO	WIRE	NOTE
3V3	HT7333 OUT and LoRa VCC	red	Power comes IN here — see page 5
GND	Battery -, HT7333 GND, LoRa GND, LED cathode	black	One common ground rail
5V	nothing	—	A 3.7 V cell cannot drive the onboard regulator
GP107	LoRa NSS	orange	Chip select
GP104	LoRa SCK	yellow	SPI clock
GP105	LoRa MISO	green	—
GP106	LoRa MOSI	blue	—
GP103	LoRa RST	purple	—
GP1010	LoRa DIO0	pink	RX-done interrupt
GP101	330 Ω → LED anode	orange	External LED, active HIGH
GP108	onboard LED	—	Active LOW on a SuperMini; no wiring needed

C3 PINS YOU MUST LEAVE ALONE

GPIO11–17 are wired to the internal flash — using one stops the chip booting. GPIO18/19 are the native USB lines. GPIO20/21 are UART0. GPIO2, 8 and 9 are strapping pins read at power-up, so nothing may hold them at the wrong level while the board resets. The map above avoids all of them.

3 · What the receiver tells you

One LED, three states, plus a threshold you can move.

What the LED tells you

LED	MEANING
One short flash, 80 ms	Packet received, level below the alert mark — the link is alive and all is quiet
ON, steady	ALERT — the level is at or above the mark and stays there
Three fast flashes	A packet arrived but was corrupted in the air — edge of range, or the antennas are too close together
Off, nothing printed	Nothing is being received at all

Where the alert mark sits

The sender reports **level**, not distance: $\text{level} = \text{DATUM_MM} - \text{distance}$. Level therefore *rises* as the target gets *closer* to the transducer — exactly as river stage rises as water comes up toward a bridge-mounted sensor. With the sender's DATUM_MM left at its bench value of 2000 mm:

HAND BELOW THE SENSOR	REPORTED LEVEL	LED
1500 mm	500 mm	short flash per packet
1000 mm	1000 mm	short flash per packet
600 mm	1400 mm	short flash per packet
500 mm or closer	1500 mm or more	solid ON — alert

CONSTANT	DEFAULT	WHAT IT DOES
ALERT_ON_MM	1500	Alert turns on at or above this level
ALERT_OFF_MM	1400	And clears only once the level falls back below this
ALERT_HOLD_MS	6000	Clear the alert if no packet arrives for this long

WHY TWO THRESHOLDS AND NOT ONE

The 100 mm gap between on and off is hysteresis. A level sitting exactly on a single threshold would switch the alarm on and off several times a second, which on a real river means a village receiving six evacuation messages in a minute. Widen the gap if your readings are noisy; never remove it.

THE ALERT CLEARS ITSELF IF THE SENDER GOES QUIET

After six seconds with no packet the receiver drops the alert and prints `alert cleared - no packets, level unknown`. A light left burning on stale data is worse than no light, because it looks like live information.

LED POLARITY

The long leg is the anode and goes to the resistor and on to GPIO25. The short leg, on the flat side of the LED body, goes to ground. Backwards, it simply stays dark — no harm done, but you will blame the radio.

4 • Powering from a cell

A single lithium cell runs either board. It cannot go where a USB supply goes, and that is the whole of what makes this different.

Why the battery cannot feed VIN

The DevKit's onboard AMS1117 regulator needs roughly **4.8 V** at its input to produce a stable 3.3 V. A lithium cell sits between 4.2 V fully charged and 3.0 V flat, so it never clears that. Wire a cell to VIN and the board will half-run: it may boot, then brown out the moment the radio transmits, and look for all the world like a firmware bug.

So the cell goes through an external low-dropout regulator instead, and its 3.3 V output is fed **directly into the 3V3 pin**, bypassing the onboard regulator entirely. The HT7333-A drops only about 100 mV, so it keeps regulating until the cell is genuinely flat at 3.4 V.

STEP	CONNECTION	NOTE
1	Cell + → switch → HT7333 IN	The switch cuts the battery for flashing
2	Cell – → HT7333 GND → ESP32 GND	One common ground for everything
3	HT7333 OUT → ESP32 3V3 pin	Never VIN, never the USB connector
4	100 µF across HT7333 OUT and GND	Bulk on the 3.3 V rail
5	470 µF at the LoRa module's own pads	Supplies the 120 mA transmit burst

DO NOT RUN USB AND BATTERY AT THE SAME TIME

With USB plugged in, the onboard AMS1117 drives the 3V3 net; the HT7333 drives the same net from the other side. Two regulators fighting over one rail is bad for both. Switch the battery OFF whenever the USB cable is connected — that is what the slide switch is for.

YOU LOSE THE SERIAL MONITOR ON BATTERY

Running on a cell, there is no USB connection and therefore nothing printing to a terminal. That is fine for the receiver, which tells you everything through its LED. For the sender, do the first run on USB to confirm the distance readings look right, and move to battery once it works.

How long a cell lasts, and other supplies

These sketches never sleep, so a 3000 mAh cell runs the sender (≈ 55 mA) about two days and the receiver (≈ 45 mA) about two and a half. The field firmware sleeps between readings and averages 7 mAh per day — the difference between two days and most of a year. Read nothing into these bench figures beyond "it will run all afternoon".

SUPPLY	WIRE IT TO	VERDICT
USB power bank	The USB socket	Simplest of all — no regulator, no wiring
4 × AA, 6 V	VIN and GND	Works — 6 V clears the onboard regulator
3 × AA, 4.5 V	VIN and GND	Marginal; browns out as the cells fade
9 V PP3 battery	VIN and GND	Works, but the regulator wastes most of it as heat and the battery dies fast
Single lithium cell	HT7333 → 3V3 pin	The circuit on page 2 and 3

THIS IS THE FIELD POWER CHAIN, MINUS THE SOLAR

The cell, the regulator and the 3V3 feed here are exactly what the deployed node uses. Adding solar means a CN3065 charge module between the panel and the cell — nothing after the cell changes.

5 • Running the test

Order of work

1. Fit both antennas. Wire both boards from the tables on pages 2 and 3, with the 470 µF capacitor at each LoRa module's own VCC and GND pads.
2. Install the **LoRa** library by Sandeep Mistry from the Arduino Library Manager. Nothing else is needed.
3. Select the boards: ESP32 Dev Module for the sender, ESP32C3 Dev Module for the receiver. On the C3, set **USB CDC On Boot: Enabled** — without it the serial monitor stays blank while the LED works perfectly.
4. Flash bench_receiver_c3.ino to one board. Open its serial monitor at 115200 baud. It should print RECEIVER ready. Listening. and then complain every ten seconds that it has heard nothing.
5. Flash bench_sender.ino to the other board, on a second USB port. Its serial monitor should start printing a distance every two seconds.
6. Watch the receiver. Its LED should flash once per transmission, and its serial monitor should print the node, sequence number, level, RSSI and SNR.
7. Wave your hand under the transducer. The distance on the sender should change, and the level on the receiver should change with it — that is the whole chain working.

Reading the numbers

FIELD	WHAT GOOD LOOKS LIKE
dist	Matches a tape measure within about 10 mm. Below 250 mm it reports -1: that is the sensor's blind zone, not a fault.
seq	Increments by exactly one each time. Gaps mean lost packets.
RSSI	On a desk, -20 to -60 dBm. Across a building, -90 to -110. Below -115 you are at the edge.
SNR	Positive on a desk. Down to -7 dB still decodes at SF9.
good / bad	bad should stay at zero indoors. Rising bad counts mean interference or too little margin.
ok / ALERT	The first word on each line. Move your hand from 1.5 m to 0.4 m below the sensor and it should switch to ALERT and the LED should go solid.

When it does not work

SYMPTOM	CAUSE	FIX
"LoRa init failed" on boot	SPI wiring, or NSS/RST/DIO0 on the wrong pins	Re-check D5, D18, D19, D23, D14, D26 against the table
Both boards run, receiver hears nothing	A radio setting differs between the two sketches	Compare frequency, SF, BW, coding rate and sync word line by line
Everything looks right, still nothing	Frequency set for the wrong chip — an SX1278 cannot reach 865 MHz	Both sketches must read 433000000L for an SX1278 board
Works at 1 m, dead at 10 m	Wrong antenna for the band	433 MHz module needs a 433 MHz antenna — the longer one
Sender reboots every two seconds	Transmit burst browning out the rail	470 µF at the module's pads; try a different USB port or cable
Distance always -1	Sensor too close to the target, or ECHO not connected	Hold it at least 300 mm from the floor; check GPIO16
Distance wildly unstable	Aimed at a soft or angled surface	Point it square at a hard flat floor or a book
Three fast flashes constantly	Antennas within a few centimetres of each other	Move the boards a metre apart — a receiver can be overloaded by too much signal
Receiver LED never lights but serial prints fine	External LED reversed	Long leg to the resistor; or just watch the onboard blue LED
C3 serial monitor stays blank, LED works	USB CDC On Boot is disabled, so Serial goes to UART0	Enable it in Tools, then re-flash
C3 will not boot, or reboots in a loop	Something loading GPIO2, 8 or 9 — they are strapping pins	Keep those three clear; the map on page 3 already does
LED solid on from the moment it boots	Sensor reading small distances, so level is already above the mark	Point the transducer at something further than 500 mm away, or raise ALERT_ON_MM
Alert never triggers however close you get	Below 250 mm the sensor returns -1 and level becomes -1	Stay outside the blind zone — 300 to 500 mm is the useful test range

ONCE THIS WORKS, THE FIELD BUILD IS MOSTLY ADDITION

The radio link, the SPI bus and the sensor are the hard parts, and this test proves all three. Adding the rain gauge, the temperature probe, the battery and deep sleep changes the node firmware but not the wiring you have just verified.

6 • Sender firmware

One file. Same pin map as the field node.

BENCH_SENDER/BENCH_SENDER.INO – PART 1 OF 3

```
// =====
// BENCH TEST – SENDER
// ESP32 + JSN-SR04T ultrasonic + SX1276 LoRa.
// No rain gauge, no temperature probe, no battery monitor, no sleep.
// Measures distance every 2 s, transmits it, blinks its own LED.
//
// The packet is the same 13-byte format the field system uses, with the
// rain and battery fields set to zero - so the real gateway firmware
// will decode these packets unchanged when you move on.
// =====

#include <SPI.h>
#include <LoRa.h>

#define NODE_ID      1
#define DATUM_MM     2000 // bench value: pretend the sensor sits 2 m up
#define TX_INTERVAL  2000 // milliseconds between transmissions
#define FIXED_TEMP_C 25.0f // no DS18B20 on the bench

#define PIN_TRIG     17
#define PIN_ECHO     16
#define PIN_LED      2 // the DevKit's own blue LED

#define LORA_SS      5
#define LORA_RST     14
#define LORA_DIO0     26

// These five must be identical on the receiver or nothing arrives.
// RA-02 carries an SX1278, which tunes 137-525 MHz. It CANNOT do 865 MHz:
// set that and the radio still initialises, it just never hears anything.
#define LORA_FREQ     433000000L // 433.0 MHz - SX1278 / RA-02
#define LORA_SF        9
#define LORA_BW       125E3
#define LORA_CR        5
#define LORA_SYNC     0x34

uint16_t seq = 0;

uint16_t crc16(const uint8_t *d, size_t n) {
    uint16_t crc = 0xFFFF;
    for (size_t i = 0; i < n; i++) {
        crc ^= (uint16_t)d[i] << 8;
        for (uint8_t b = 0; b < 8; b++)
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
    }
    return crc;
}
```

```

uint32_t pingOnce() {
    digitalWrite(PIN_TRIG, LOW); delayMicroseconds(4);
    digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);
    return pulseIn(PIN_ECHO, HIGH, 40000UL);    // 40 ms ceiling, about 6.8 m
}

int cmpU32(const void *a, const void *b) {
    uint32_t x = *(const uint32_t*)a, y = *(const uint32_t*)b;
    return (x > y) - (x < y);
}

// Median of 9 pings. Returns millimetres, or -1 if the reading is no good.
int16_t measureDistanceMM() {
    uint32_t s[9];
    uint8_t n = 0;
    for (uint8_t i = 0; i < 9; i++) {
        uint32_t t = pingOnce();
        if (t > 0) s[n++] = t;
        delay(40);
    }
    if (n < 5) return -1;
    qsort(s, n, sizeof(uint32_t), cmpU32);

    float c = 331.4f + 0.606f * FIXED_TEMP_C;    // speed of sound, m/s
    float mm = (s[n / 2] * c) / 2000.0f;
    if (mm < 250.0f || mm > 4500.0f) return -1;  // blind zone / out of range
    return (int16_t)(mm + 0.5f);
}

void sendPacket(int16_t levelMM) {
    uint8_t p[13];
    p[0] = 1;                                // protocol version
    p[1] = NODE_ID;
    p[2] = seq >> 8;    p[3] = seq & 0xFF;
    p[4] = levelMM >> 8; p[5] = levelMM & 0xFF;
    p[6] = 0;    p[7] = 0;    // rain tips - none on the bench
    p[8] = 0;    p[9] = 0;    // battery mV - USB powered
    p[10] = (int8_t)FIXED_TEMP_C;
    uint16_t c = crc16(p, 11);
    p[11] = c >> 8;    p[12] = c & 0xFF;

    LoRa.beginPacket();
    LoRa.write(p, 13);
    LoRa.endPacket();    // blocks until the packet is on the air
    seq++;
}

void setup() {
    Serial.begin(115200);
    delay(300);
    pinMode(PIN_TRIG, OUTPUT);
    pinMode(PIN_ECHO, INPUT);
    pinMode(PIN_LED, OUTPUT);
    digitalWrite(PIN_LED, LOW);

    SPI.begin(18, 19, 23, LORA_SS);
    LoRa.setPins(LORA_SS, LORA_RST, LORA_DI00);
    while (!LoRa.begin(LORA_FREQ)) {
        Serial.println("LoRa init failed - check wiring, and fit a 433 MHz antenna");
        delay(2000);
    }
    LoRa.setSpreadingFactor(LORA_SF);
    LoRa.setSignalBandwidth(LORA_BW);
    LoRa.setCodingRate4(LORA_CR);
    LoRa.setSyncWord(LORA_SYNC);
    LoRa.setTxPower(14);

    Serial.println("SENDER ready. Transmitting every 2 s.");
}

```

```
void loop() {
  int16_t dist = measureDistanceMM();
  int16_t level = (dist < 0) ? -1 : (int16_t)(DATUM_MM - dist);

  Serial.print("seq "); Serial.print(seq);
  Serial.print("  dist "); Serial.print(dist);
  Serial.print(" mm  level "); Serial.print(level);
  Serial.println(" mm");

  sendPacket(level);

  digitalWrite(PIN_LED, HIGH);    // one flash per transmission
  delay(60);
  digitalWrite(PIN_LED, LOW);

  delay(TX_INTERVAL);
}
```

7 • Receiver firmware

Listens, checks the CRC, flashes the LED, prints the packet.

BENCH_RECEIVER_C3/BENCH_RECEIVER_C3.INO – PART 1 OF 3

```
// =====
// BENCH TEST - RECEIVER (ESP32-C3)
// ESP32-C3 + RA-02 / SX1278 LoRa at 433 MHz. No WiFi, no MQTT, no SD.
//
// Receives level packets from the classic-ESP32 sender, prints them,
// and drives one LED:
//
//   one short flash (80 ms) = packet received, level below the mark
//   LED ON, steady         = ALERT - level at or above the mark
//   three fast flashes     = packet arrived but the CRC failed
//   LED off, nothing printed = nothing being received
//
// -----
// ARDUINO IDE SETTINGS (these matter more than usual on the C3)
// Board ..... "ESP32C3 Dev Module"
// USB CDC On Boot ..... ENABLED <-- or Serial prints nowhere
// Upload Speed ..... 921600
// The C3 talks to the PC over native USB, not a USB-serial chip. With
// USB CDC On Boot disabled, Serial goes to UART0 on GPIO20/21 instead
// and the monitor stays blank while the LED works perfectly.
// -----
// =====

#include <SPI.h>
#include <LoRa.h>

// ----- pins: ESP32-C3 -----
// Go by GPIO NUMBER, not by position - C3 boards vary physically.
// Avoided on purpose:
//   GPIO11-17 wired to the internal flash. Using them bricks the boot.
//   GPIO18/19 native USB D-/D+.
//   GPIO20/21 UART0.
//   GPIO28/9 strapping pins, read at power-up.
#define LORA_SCK 4
#define LORA_MISO 5
#define LORA_MOSI 6
#define LORA_SS 7
#define LORA_RST 3
#define LORA_DIO0 10

#define PIN_LED 1 // external LED + 330 ohm to GND, active HIGH
#define PIN_LED_ONBOARD 8 // SuperMini's blue LED, wired ACTIVE LOW
#define HAS_ONBOARD_LED 1 // set 0 if your board has none, or an RGB one

// ----- radio: identical to the sender or nothing arrives -----
#define LORA_FREQ 433000000L // 433.0 MHz - SX1278 / RA-02
#define LORA_SF 9
#define LORA_BW 125E3
#define LORA_CR 5
#define LORA_SYNC 0x34
```

```

// ----- the alert threshold -----
// The sender reports LEVEL, not distance: level = DATUM_MM - distance
// so level RISES as the target gets CLOSER to the transducer, exactly as
// river stage rises as water comes up toward a bridge-mounted sensor.
//
// With the sender's DATUM_MM = 2000:
//   hand 1500 mm away -> level 500 mm   quiet
//   hand 500 mm away  -> level 1500 mm  ALERT
#define ALERT_ON_MM 1500
#define ALERT_OFF_MM 1400      // hysteresis: must fall below this to clear
#define ALERT_HOLD_MS 6000    // clear the alert if nothing is heard

unsigned long ledOffAt = 0;
unsigned long lastHeard = 0;
unsigned long lastWarn = 0;
uint32_t goodCount = 0, badCount = 0;
bool alert = false;

uint16_t crc16(const uint8_t *d, size_t n) {
  uint16_t crc = 0xFFFF;
  for (size_t i = 0; i < n; i++) {
    crc ^= (uint16_t)d[i] << 8;
    for (uint8_t b = 0; b < 8; b++)
      crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
  }
  return crc;
}

void ledWrite(bool on) {
  digitalWrite(PIN_LED, on ? HIGH : LOW);      // external: active HIGH
#ifdef HAS_ONBOARD_LED
  digitalWrite(PIN_LED_ONBOARD, on ? LOW : HIGH); // onboard: active LOW
#endif
}

void flashGood() {
  // one short flash, does not block reception
  ledWrite(true);
  ledOffAt = millis() + 80;
}

void flashBad() {
  // three fast flashes
  for (uint8_t i = 0; i < 3; i++) {
    ledWrite(true); delay(50);
    ledWrite(false); delay(50);
  }
}

void setup() {
  Serial.begin(115200);
  delay(400);      // native USB needs a moment to enumerate

  pinMode(PIN_LED, OUTPUT);
#ifdef HAS_ONBOARD_LED
  pinMode(PIN_LED_ONBOARD, OUTPUT);
#endif
  ledWrite(false);

  SPI.begin(LORA_SCK, LORA_MISO, LORA_MOSI, LORA_SS);
  LoRa.setPins(LORA_SS, LORA_RST, LORA_DI00);
  while (!LoRa.begin(LORA_FREQ)) {
    Serial.println("LoRa init failed - check wiring, and fit a 433 MHz antenna");
    delay(2000);
  }
  LoRa.setSpreadingFactor(LORA_SF);
  LoRa.setSignalBandwidth(LORA_BW);
  LoRa.setCodingRate4(LORA_CR);
  LoRa.setSyncWord(LORA_SYNC);
  LoRa.receive();      // park in continuous receive

  Serial.println("RECEIVER ready (ESP32-C3). Listening on 433.0 MHz.");
  lastHeard = millis();
}

```

```

void loop() {
  // while the alert is on the LED stays lit; otherwise end the short flash
  if (!alert && ledOffAt && millis() > ledOffAt) { ledWrite(false); ledOffAt = 0; }

  // a stuck alert is worse than no alert: clear it if the sender goes quiet
  if (alert && millis() - lastHeard > ALERT_HOLD_MS) {
    alert = false;
    ledWrite(false);
    Serial.println("alert cleared - no packets, level unknown");
  }

  // say something if the link has gone quiet
  if (millis() - lastHeard > 10000 && millis() - lastWarn > 10000) {
    lastWarn = millis();
    Serial.println("...nothing for 10 s. Sender powered? Same radio settings?");
  }

  int sz = LoRa.parsePacket();
  if (sz == 0) return;

  if (sz != 13) { // not one of ours
    Serial.print("ignored a packet of "); Serial.print(sz); Serial.println(" bytes");
    return;
  }

  uint8_t p[13];
  for (int i = 0; i < 13; i++) p[i] = LoRa.read();

  uint16_t got = ((uint16_t)p[11] << 8) | p[12];
  if (got != crc16(p, 11)) {
    badCount++;
    Serial.println("CRC FAILED - packet corrupted in the air");
    flashBad();
    return;
  }

  uint8_t node = p[1];
  uint16_t seq = ((uint16_t)p[2] << 8) | p[3];
  int16_t level = (int16_t)(((uint16_t)p[4] << 8) | p[5]);

  goodCount++;
  lastHeard = millis();

  // ----- the alert decision, with hysteresis -----
  if (!alert && level >= ALERT_ON_MM) {
    alert = true;
    ledWrite(true); // solid on, stays on
    Serial.println("*** ALERT - level above the mark ***");
  } else if (alert && level < ALERT_OFF_MM) {
    alert = false;
    ledWrite(false);
    Serial.println("alert cleared - level back below the mark");
  }
  if (!alert) flashGood();

  Serial.print(alert ? "ALERT " : "ok ");
  Serial.print("node "); Serial.print(node);
  Serial.print(" seq "); Serial.print(seq);
  Serial.print(" level "); Serial.print(level);
  Serial.print(" mm RSSI "); Serial.print(LoRa.packetRssi());
  Serial.print(" dBm SNR "); Serial.print(LoRa.packetSnr(), 1);
  Serial.print(" dB good "); Serial.print(goodCount);
  Serial.print(" / bad "); Serial.println(badCount);
}

```